

Proyecto Final 2026-1

Motor de Búsqueda en Texto Comprimido: FM-Index y Wavelet Trees

Documento técnico de referencia para el equipo de desarrollo — Universidad de Ingeniería y Tecnología · UTEC · 2026

1.	El problema base	2
2.	Aplicación real: el genoma humano	2
3.	Glosario de componentes	3
4.	Suffix Array — definición y construcción	4
5.	BWT — Burrows-Wheeler Transform	5
6.	Tabla C	6
7.	Wavelet Tree sobre la BWT	6
8.	FM-Index: el sistema completo	8
9.	Ejemplo completo con queries	8
		1
10.	Análisis de complejidad	2

1. El problema base

El problema fundamental que resuelve este proyecto es el siguiente:

Problema de búsqueda de subcadenas: Dado un texto T de longitud n y un patrón P de longitud m , encontrar todas las posiciones de T donde P aparece como subcadena, de la forma más eficiente posible en tiempo y espacio.

Ejemplo mínimo:

ENUNCIADO

$T = \text{G A T A G A C A \$}$

0 1 2 3 4 5 6 7 8

$P = \text{G A}$

Resultado esperado: P aparece en las posiciones 0 y 4.

La solución naïve con dos bucles anidados recorre T completo por cada posición, resultando en $O(n \cdot m)$ comparaciones. Para $n = 3,200,000,000$ y $m = 150$, eso es 480 mil millones de operaciones por consulta — inviable cuando hay millones de consultas diarias. La necesidad de una solución eficiente es el punto de partida de todo lo que viene a continuación.

2. Aplicación real: el genoma humano

¿Qué es el ADN y el genoma?

Cada célula del cuerpo humano contiene en su núcleo una molécula llamada ADN. El ADN es el manual de instrucciones biológico: le indica a cada célula qué proteínas fabricar, cómo dividirse y cómo funcionar. Físicamente está formado por una cadena de cuatro moléculas llamadas bases nitrogenadas, que abreviamos con una sola letra cada una:

A	C	G	T
Adenina	Citosina	Guanina	Timina

El genoma humano completo es una cadena de aproximadamente **3,200,000,000** de estas letras. Si lo escribieras como texto se vería así:

GENOMA HUMANO (fragmento)

...ACGTTAGCTAGCTTGCAAACGTTAGCGCTAGCTTAG...

(3,200,000,000 caracteres del alfabeto {A, C, G, T})

¿Por qué sirve para detectar enfermedades?

El genoma de dos personas distintas es idéntico en un 99.9%. El 0.1% restante contiene las diferencias que nos hacen únicos: predisposición a enfermedades, respuesta a medicamentos, rasgos físicos. Un cambio en una sola posición del genoma (por ejemplo, una A donde debería haber una G) puede causar una enfermedad genética o hacer que un medicamento sea ineficaz para un paciente.

Para detectar esas diferencias se compara el genoma del paciente contra un **genoma de referencia** estándar. El problema es que las máquinas de secuenciación modernas no leen el genoma de corrido: lo fragmentan en millones de pedazos cortos llamados **reads**, de entre 100 y 300 letras cada uno. Para reconstruir el genoma del paciente hay que encontrar en qué posición del genoma de referencia encaja cada read. Eso es exactamente el problema de búsqueda de subcadenas, a escala masiva:

T = Genoma de referencia	P = Cada read del paciente
n = 3,200,000,000 caracteres	m = ~150 caracteres
Se construye una sola vez	Se hacen ~30,000,000 consultas por experimento

Las mismas herramientas tienen aplicación en: motores de búsqueda de texto, detección de plagio, compresión de datos, y bases de datos de proteínas. El genoma es simplemente el caso de uso más exigente en escala.

3. Glosario de componentes del sistema

El sistema completo que implementaremos se llama **FM-Index** (Ferragina-Manzini Index, 2000). Está compuesto por cuatro piezas que se construyen en orden y se usan juntas para la búsqueda. Las siguientes secciones definen cada una en detalle.

Suffix Array (SA)	Array de n enteros que representa todos los sufijos del texto T ordenados alfabéticamente. Cada entero es la posición de inicio del sufijo en T. Base de todo el sistema.
BWT — Burrows-Wheeler Transform	Transformación del texto que produce una permutación de sus caracteres con la propiedad de que letras iguales se agrupan. Se calcula en $O(n)$ a partir del SA. Es el string sobre el que opera el FM-Index.
Tabla C	Array de sigma enteros (uno por carácter del alfabeto). $C[c]$ = cuántos caracteres del texto son lexicográficamente menores que c. Ocupa $O(\text{sigma})$ espacio y se usa en cada paso del backward search.

Wavelet Tree	Estructura de datos construida sobre la BWT. Permite responder $\text{rank}(i, c)$ — cuántas veces aparece el carácter c en $\text{BWT}[0..i-1]$ — en $O(\log \sigma)$ tiempo y $n \cdot \log \sigma$ bits de espacio. Generaliza rank/select a alfabetos de tamaño arbitrario.
Backward Search	Algoritmo de consulta del FM-Index. Lee el patrón P de derecha a izquierda y en cada paso usa C y rank (via Wavelet Tree) para reducir un rango $[lo, hi]$ en el SA. El rango final indica cuántas y cuáles ocurrencias tiene P en T . Tiempo total: $O(m \cdot \log \sigma)$.

4. Suffix Array

Definición formal

Dado el texto $T = T[0]T[1]...T[n-1]\$$ (donde $\$$ es un carácter terminal menor que todos los demás), el sufijo S_i es la subcadena $T[i..n-1]\$$. El Suffix Array SA es el arreglo de $n+1$ enteros tal que $SA[i] = j$ si y solo si S_j es el i -ésimo sufijo en orden lexicográfico.

Construcción sobre $T = \text{GATAGACA}\$$

Paso 1 — Listar todos los sufijos:

SUFIJOS (no se almacenan completos — solo los índices de inicio)

$T = \text{G A T A G A C A \$}$

0 1 2 3 4 5 6 7 8

pos 0 -> GATAGACA\$

pos 1 -> ATAGACA\$

pos 2 -> TAGACA\$

pos 3 -> AGACA\$

pos 4 -> GACA\$

pos 5 -> ACA\$

pos 6 -> CA\$

pos 7 -> A\$

pos 8 -> \$

Importante: el SA no guarda los sufijos completos — eso ocuparía $O(n^2)$ espacio. Solo guarda el índice de inicio de cada sufijo. Para comparar dos sufijos durante la construcción se accede al texto original en $O(1)$ por carácter.

Paso 2 — Ordenar alfabéticamente (orden: $\$ < A < C < G < T$):

SUFFIX ARRAY RESULTANTE

i=0 SA[0]=8 -> \$

i=1 SA[1]=7 -> A\$

i=2 SA[2]=5 -> ACA\$

i=3 SA[3]=3 -> AGACA\$

i=4 SA[4]=1 -> ATAGACA\$

i=5 SA[5]=6 -> CA\$

```
i=6 SA[6]=4 -> GACA$  
i=7 SA[7]=0 -> GATAGACA$  
i=8 SA[8]=2 -> TAGACA$  
  
SA = [ 8, 7, 5, 3, 1, 6, 4, 0, 2 ]
```

Heurística clave: Si el patron P aparece en T, todos los sufijos que empiezan con P quedan en un bloque contiguo del SA ordenado. Buscar ese bloque con busqueda binaria toma $O(m \cdot \log n)$ en lugar de $O(n \cdot m)$ del metodo naive. El FM-Index bajara ese costo aun mas, a $O(m \cdot \log \sigma)$.

La construccion naive del SA (generar sufijos y ordenar con sort estandar) cuesta $O(n^2 \log n)$ porque cada comparacion entre dos sufijos puede requerir hasta $O(n)$ pasos. Los algoritmos especializados SA-IS y DC3 lo reducen a $O(n)$ explotando la estructura compartida entre sufijos del mismo texto.

5. Burrows-Wheeler Transform (BWT)

Definicion formal

Dado el Suffix Array SA del texto T, la BWT se define como:

FORMULA DE LA BWT

$BWT[i] = T[SA[i] - 1]$

Caso especial: si $SA[i] = 0 \rightarrow BWT[i] = T[n-1]$ (ultimo caracter, el \$)

Es decir: $BWT[i]$ es el caracter que esta justo antes del sufijo que ocupa la posicion i en el SA ordenado.

Calculo sobre T = GATAGACA\$

CALCULO BWT

```
i=0 SA[0]=8 -> T[8-1] = T[7] = 'A'  
i=1 SA[1]=7 -> T[7-1] = T[6] = 'C'  
i=2 SA[2]=5 -> T[5-1] = T[4] = 'G'  
i=3 SA[3]=3 -> T[3-1] = T[2] = 'T'  
i=4 SA[4]=1 -> T[1-1] = T[0] = 'G'  
i=5 SA[5]=6 -> T[6-1] = T[5] = 'A'  
i=6 SA[6]=4 -> T[4-1] = T[3] = 'A'  
i=7 SA[7]=0 -> caso especial -> T[8] = '$'
```

```
i=8 SA[8]=2 -> T[2-1] = T[1] = 'A'
```

```
BWT = A C G T G A A $ A
```

```
0 1 2 3 4 5 6 7 8
```

Tabla F y L (columnas primera y ultima):

MATRIZ F|L (F se obtiene gratis: caracteres de T ordenados)

i F (1ra col) L=BWT (ult col) SA[i] sufijo

```
0 $ A 8 $
```

```
1 A C 7 A$
```

```
2 A G 5 ACA$
```

```
3 A T 3 AGACA$
```

```
4 A G 1 ATAGACA$
```

```
5 C A 6 CA$
```

```
6 G A 4 GACA$
```

```
7 G $ 0 GATAGACA$
```

```
8 T A 2 TAGACA$
```

Por que la BWT agrupa caracteres iguales: Los sufijos que comparten prefijo quedan juntos en el SA. El caracter que precede a esos sufijos en el texto tiende a ser el mismo (porque todos pertenecen al mismo contexto). En GATAGACA\$ las A's aparecen juntas en posiciones 1-4 del SA, y sus caracteres previos forman el bloque C G T G de la BWT. En el genoma real con millones de repeticiones, esto produce runs de miles de caracteres identicos — lo que hace la BWT altamente compresible.

6. Tabla C

$C[c]$ es el numero de caracteres del texto T que son estrictamente menores que c en orden lexicografico. Dicho de otra forma: $C[c]$ es la primera posicion en la columna F del SA donde aparece el caracter c.

Calculo a partir del conteo de caracteres en T = GATAGACA\$:

CALCULO TABLA C — $O(n)$ tiempo, $O(\sigma)$ espacio

Conteo en T: \$->1 A->4 C->1 G->2 T->1

$C[\$] = 0$ (nada es menor que \$)

$C[A] = 1$ (1 dolar antes)

$C[C] = 1 + 4 = 5$ (1 dolar + 4 A's)

$C[G] = 1 + 4 + 1 = 6$ (+ 1 C)

$C[T] = 1+4+1+2 = 8$ (+ 2 G's)

$C = \{ \$:0, A:1, C:5, G:6, T:8 \}$

Uso en el backward search: $C[c]$ te dice inmediatamente en que fila del SA empieza el bloque de sufijos que comienzan con c, sin necesidad de buscar. En cada iteracion del backward search se usa $C[c]$ como desplazamiento base para calcular el nuevo rango $[lo, hi]$.

7. Wavelet Tree sobre la BWT

¿Por que un Wavelet Tree y no otra cosa?

El backward search necesita la operacion $\text{rank}(i, c)$: cuantas veces aparece el caracter c en $\text{BWT}[0..i-1]$. Para alfabeto binario $\{0,1\}$, existe una estructura succinct que responde rank en $O(1)$ con $O(n)$ bits de espacio (cubierta en el bloque 5.8 del silabo). Para alfabeto de tamaño $\sigma > 2$, las alternativas directas son malas:

sigma bitvectores separados (uno por caracter)	Wavelet Tree
Espacio: $\sigma \times n$ bits	Espacio: $n \times \log(\sigma)$ bits
Para $\sigma=256$: 256 veces el texto original	Para $\sigma=4$: solo $2n$ bits Para $\sigma=256$: $8n$ bits

Construccion — BWT = A C G T G A A \$ A

Se construye un arbol binario de altura $\log(\sigma)$ niveles. En cada nodo se guarda un bitvector que indica si cada caracter del subnivel va a la mitad izquierda (bit 0) o derecha (bit 1) del alfabeto restante.

Nivel 0 — Raiz, alfabeto $\{\$,A,C,G,T\}$:

Mitad izquierda = $\{\$,A,C\}$ (bit 0) | Mitad derecha = $\{G,T\}$ (bit 1)

NIVEL 0

BWT: A C G T G A A \$ A

bit: 0 0 1 1 1 0 0 0 0

$B0 = [0, 0, 1, 1, 1, 0, 0, 0, 0]$

-> Hijo izquierdo recibe: A C A A \$ A (las posiciones con bit 0)

-> Hijo derecho recibe: G T G (las posiciones con bit 1)

Nivel 1 — Hijo izquierdo, alfabeto $\{\$,A,C\}$:

Mitad izquierda = $\{\$,A\}$ (bit 0) | Mitad derecha = $\{C\}$ (bit 1)

NIVEL 1 — HIJO IZQUIERDO

Seq: A C A A \$ A

bit: 0 1 0 0 0 0

$B1L = [0, 1, 0, 0, 0, 0]$

```
-> Hijo izq recibe: A A $ A (hoja – solo {$,A})  
-> Hijo der recibe: C (hoja – solo {C})
```

Nivel 1 — Hijo derecho, alfabeto {G,T}:

Mitad izquierda = {G} (bit 0) | Mitad derecha = {T} (bit 1)

NIVEL 1 — HIJO DERECHO

Seq: G T G

bit: 0 1 0

B1R = [0, 1, 0]

-> Hijo izq recibe: G G (hoja)

-> Hijo der recibe: T (hoja)

Por que se usan bits: En lugar de guardar el caracter directamente (que requiere $\log(\sigma)$ bits por posicion), se codifica cada caracter distribuyendolo por niveles del arbol: en cada nivel solo se necesita saber si va a la izquierda o derecha — eso es 1 bit. El total es exactamente n bits por nivel $\times \log(\sigma)$ niveles = $n \times \log(\sigma)$ bits, el minimo teorico para representar el string. Ademias, cada bitvector admite la estructura succinct de rank/select en $O(1)$ del silabo (bloque 5.8).

Operacion rank(i, c) con el Wavelet Tree:

Ejemplo: rank(5, G) — cuantas G hay en BWT[0..4]

EJEMPLO rank(5, G)

G esta en la mitad derecha {G,T} -> bit 1 en la raiz

Nivel 0: cuantos 1s hay en B0[0..4]?

B0 = [0,0,1,1,1,0,0,0,0] -> en posiciones 0..4: [0,0,1,1,1] -> 3 unos

Nuevo indice para hijo derecho: 3

G esta en la mitad izquierda {G} del hijo derecho -> bit 0

Nivel 1R: cuantos 0s hay en B1R[0..2]?

B1R = [0,1,0] -> en posiciones 0..2: [0,1,0] -> 2 ceros

Llegamos a la hoja {G}. rank(5, G) = 2 ✓

Verificacion manual: BWT[0..4] = A C G T G -> hay 2 G's. Correcto.

Cada operacion rank baja por $\log(\sigma)$ niveles del arbol, haciendo un rank binario $O(1)$ en cada nivel. Tiempo total por operacion rank: $O(\log \sigma)$. Para el genoma con $\sigma=4$: $\log(4)=2$ niveles, 2 operaciones elementales por rank.

8. FM-Index: el sistema completo

El FM-Index es la combinacion de SA + BWT + Tabla C + Wavelet Tree. Una vez construido, la busqueda usa el algoritmo de Backward Search:

```
ALGORITMO CORE —  $O(m \cdot \log \sigma)$  por consulta
BACKWARD SEARCH — pseudocodigo

funcion buscar(P, C, BWT_WT):
    lo = 0
    hi = n (longitud del texto + 1)

    para i desde len(P)-1 hasta 0: // leer P de derecha a izquierda
        c = P[i]
        lo = C[c] + rank(lo, c) // rank sobre el Wavelet Tree
        hi = C[c] + rank(hi, c)

    si lo >= hi:
        retornar 'P no aparece en T'

    retornar hi - lo // numero de ocurrencias

    // posiciones exactas: SA[lo], SA[lo+1], ..., SA[hi-1]
```

9. Ejemplo completo con dos queries

Usaremos $T = \text{GATAGACA\$}$ con el SA, BWT, C y Wavelet Tree ya construidos en las secciones anteriores. Ejecutamos dos queries: una donde el patron aparece y otra donde no.

Indice construido (referencia rapida):

```
INDICE FM

T = G A T A G A C A $ (posiciones 0..8)

SA = [ 8, 7, 5, 3, 1, 6, 4, 0, 2 ]

BWT = A C G T G A A $ A (posiciones 0..8)

F = $ A A A A C G G T (columna primera del SA ordenado)

C = { $:0, A:1, C:5, G:6, T:8 }

Wavelet Tree sobre BWT:

B0 = [0,0,1,1,1,0,0,0,0] (raiz: {$,A,C}->0, {G,T}->1)
```

B1L = [0,1,0,0,0,0] (hijo izq: {\$,A}->0, {C}->1)

B1R = [0,1,0] (hijo der: {G}->0, {T}->1)

Query 1 — Buscar 'GA' (aparece en T)

init. Estado inicial	P = 'GA' -> leemos de derecha a izquierda: primero A, luego G Rango inicial: [lo, hi] = [0, 9] (todo el SA)
1. Iteracion 1 — caracter 'A' (ultimo de 'GA')	Formula: $lo = C[A] + rank(lo=0, A) = 1 + 0 = 1$ $hi = C[A] + rank(hi=9, A) = 1 + 4 = 5$ rank(0, A): rango BWT[0..-1] vacio -> 0 rank(9, A): A aparece en BWT en posiciones 0,5,6,8 -> 4 A's en toda la BWT Nuevo rango: [1, 5] Verificacion: SA[1..4] = {7,5,3,1} -> sufijos A\$, ACA\$, AGACA\$, ATAGACA\$ Todos empiezan con 'A'. Correcto.
2. Iteracion 2 — caracter 'G' (primero de 'GA')	Formula: $lo = C[G] + rank(lo=1, G) = 6 + 0 = 6$ $hi = C[G] + rank(hi=5, G) = 6 + 2 = 8$ rank(1, G): BWT[0..0] = 'A' -> 0 G's rank(5, G): BWT[0..4] = 'A C G T G' -> 2 G's (calcula via Wavelet Tree para rank(5,G)): Nivel 0: B0[0..4]=[0,0,1,1,1] -> 3 unos -> bajar a hijo der con idx=3 Nivel 1R: B1R[0..2]=[0,1,0] -> 2 ceros -> resultado: 2 Nuevo rango: [6, 8]
fin. Resultado Query 1	Rango final: [6, 8] -> hi - lo = 2 ocurrencias Posiciones: SA[6]=4 y SA[7]=0 GA aparece en T en las posiciones 0 y 4. Verificacion: GATAGACA\$ pos 0: GA TAGACA\$ ✓ pos 4: GATA GA CA\$ ✓

Query 2 — Buscar 'TC' (no aparece en T)

init. Estado inicial	P = 'TC' -> leemos de derecha a izquierda: primero C, luego T Rango inicial: [lo, hi] = [0, 9]
1. Iteracion 1 — caracter 'C' (ultimo de 'TC')	Formula: $lo = C[C] + rank(0, C) = 5 + 0 = 5$ $hi = C[C] + rank(9, C) = 5 + 1 = 6$ $rank(0, C) = 0$ (rango vacio) $rank(9, C)$: BWT completa = A C G T G A A \$ A -> 1 C -> resultado 1 Nuevo rango: [5, 6] Verificacion: SA[5]=6 -> sufijo 'CA\$'. Unica C del texto. Correcto.
2. Iteracion 2 — caracter 'T' (primero de 'TC')	Formula: $lo = C[T] + rank(5, T) = 8 + 1 = 9$ $hi = C[T] + rank(6, T) = 8 + 1 = 9$ $rank(5, T)$: BWT[0..4] = A C G T G -> 1 T -> resultado 1 $rank(6, T)$: BWT[0..5] = A C G T G A -> 1 T -> resultado 1 Nuevo rango: [9, 9] -> $lo \geq hi$ (rango vacio)
fin. Resultado Query 2	Rango final: [9, 9] -> $lo \geq hi$ -> rango vacio 'TC' NO aparece en T. Interpretacion: la unica T del texto (posicion 2) va seguida de 'A', no de 'C'. El algoritmo lo detecto sin recorrer el texto. Verificacion: GATAGACA\$ -> pos 2: T A GACA\$ ✓ (sigue A, no C)

10. Analisis de complejidad

Componente	Espacio	Construccion	Consulta
Texto original T	n bytes	—	O(1) acceso
Suffix Array SA	n x 4 bytes	O(n) con SA-IS	O(1) acceso
BWT	n bytes	O(n) dado SA	—
Tabla C	sigma enteros	O(n)	O(1) lookup
Wavelet Tree	n x log(sigma) bits	O(n log sigma)	O(log sigma) por rank
TOTAL FM-Index	O(n log sigma) bits	O(n log sigma)	O(m log sigma)
SA sin comprimir	O(n log n) bits	O(n log n) naive	O(m log n)

Ganancia concreta para el genoma humano ($\sigma=4$, $n=3.2 \times 10^9$, $m=150$): El Wavelet Tree ocupa $n \times \log(4) = n \times 2$ bits = 800 MB, frente a los 12.8 GB del SA clasico sin comprimir. Reduccion de espacio: 16x. El tiempo de busqueda por read pasa de $O(m \log n) = 150 \times 31 = 4,650$ operaciones a $O(m \log \sigma) = 150 \times 2 = 300$ operaciones. Aceleracion: ~15x por consulta. Con 30 millones de reads por experimento, esa diferencia es la que separa procesar una muestra en horas vs en dias.